

This Python code implements a **Limit Order Book (LOB)**, which is the core engine used by financial exchanges (like the NYSE or NASDAQ) to match buy and sell orders. It uses **price-time priority**, meaning orders are filled based on the best price first, and if prices are equal, the order that arrived first takes precedence.

1. The Order Data Class

The Order class represents a single instruction to buy or sell.

```
@total_ordering
@dataclass
class Order:
    timestamp: float
    order_id: str
    side: str # 'B' for bid, 'A' for ask
    price: float
    qty: int
    remaining_qty: int = field(init=False)

    def __post_init__(self):
        self.remaining_qty = self.qty
        if self.side == 'B':
            self._priority_price = -self.price # Max-heap for bids
        else:
            self._priority_price = self.price # Min-heap for asks
        self._priority_key = (self._priority_price, self.timestamp)
```

Explanation

- **@total_ordering and __lt__**: This allows the heapq module to compare orders.
- **Priority Logic**:
 - **Bids (Buy)**: We want the **highest** price first. Since Python's heapq is a min-heap, we negate the price (-self.price) so that the highest bid becomes the "smallest" value in the heap.
 - **Asks (Sell)**: We want the **lowest** price first, so we keep the price positive.
- **Time Priority**: By including self.timestamp in the _priority_key, the code ensures that if two orders have the same price, the one with the earlier timestamp is processed first.

2. The Trade Data Class

This is a simple record-keeping structure.

```
@dataclass
class Trade:
```

```
timestamp: float
buy_order_id: str
sell_order_id: str
price: float
qty: int
```

Explanation

Whenever a match is found between a buyer and a seller, a Trade object is created to log the event, recording who traded with whom, at what price, and the quantity exchanged.

3. The OrderBook Class: Data Structures

This is the "brain" of the system.

```
class OrderBook:
    def __init__(self):
        self.bids: List[Order] = [] # Max-heap
        self.asks: List[Order] = [] # Min-heap
        self.orders: Dict[str, Order] = {} # Quick lookup
        self.bid_levels: Dict[float, int] = defaultdict(int) # Price level tracking
        self.ask_levels: Dict[float, int] = defaultdict(int)
```

Explanation

- **self.bids & self.asks:** These are the priority queues (heaps) that keep the best prices at the top.
- **self.orders:** A dictionary that allows O(1) lookup of any order by its ID, which is essential for cancellations.
- **self.bid_levels & self.ask_levels:** These aggregate the total quantity available at every price point, which is used to display "Market Depth."

4. The Matching Logic (add_order)

This is the most critical part of the code. It attempts to "fill" an incoming order against existing orders.

```
def add_order(self, order: Order) -> List[Trade]:
    # ... logic for Bids ...
    while self.asks and order.remaining_qty > 0:
        best_ask = self.asks[0]
        if order.price >= best_ask.price: # Crossing the spread
            trade_qty = min(order.remaining_qty, best_ask.remaining_qty)
            # ... Record Trade ...
```

```
order.remaining_qty -= trade_qty
best_ask.remaining_qty -= trade_qty
```

Explanation

1. **Check for a Match:** If a Buy order (Bid) comes in with a price higher than or equal to the lowest Sell order (Ask), a trade occurs.
2. **Calculate Quantity:** The trade volume is the minimum of what the buyer wants and what the seller offers.
3. **Execute:** It updates the remaining_qty for both orders. If an existing order is fully filled (remaining_qty == 0), it is popped off the heap.
4. **Passive Placement:** If the incoming order isn't fully filled by existing orders, the remainder is added to the heap to wait for a future counter-party.

5. Lazy Deletion and Cleanup

Because it is difficult to remove an item from the middle of a heapq efficiently, this code uses **Lazy Deletion**.

```
def _cleanup_empty_orders(self, heap: List[Order], is_bid: bool):
    while heap and heap[0].remaining_qty == 0:
        order = heapq.heappop(heap)
        # ... remove from tracking ...
```

Explanation

When an order is cancelled or filled, the code simply sets its remaining_qty to 0. The _cleanup_empty_orders function then checks the **top** of the heap. If the best order is "empty" (qty is 0), it pops it. It continues doing this until a valid order is at the top.

6. Market Depth and Reporting

The get_top_levels and print_depth functions provide a visual representation of the book.

```
def get_top_levels(self, levels: int = 5):
    sorted_bids = sorted(self.bid_levels.items(), key=lambda x: x[0], reverse=True)
    sorted_asks = sorted(self.ask_levels.items(), key=lambda x: x[0])
    # ... returns formatted dict ...
```

Explanation

- It sorts the aggregated price levels to show the "Best Bid" (highest) and "Best Ask" (lowest).

- It calculates the **Spread**: the difference between the best bid and best ask. A narrow spread usually indicates a liquid market.

Summary of Workflow

1. **Input**: An order arrives (Buy/Sell, Price, Qty).
2. **Matching**: The engine checks the opposite side of the book.
3. **Execution**: If prices overlap, Trade records are generated.
4. **Resting**: Any unfilled quantity is stored in the bids or asks heaps.
5. **Output**: The market state and executed trades are printed.

Complexity Note

- **Adding an Order**: $O(N \log N)$ in the worst case (due to matching and heap operations), but typically very fast ($O(\log N)$).
- **Cancelling an Order**: $O(1)$ for the logic, with $O(\log N)$ cleanup if the order was at the top.